Exp no - 12a
Date - 12/09/2026
Name - Andryan Durai Jebin

IMPLEMENT ECHO CLIENT-SERVER USING TCP/UDP SOCKETS

AIM:

To implement an echo client-server application using TCP/UDP sockets and understand end-to-end communication at the transport layer.

ALGORITHM:

1. Start the server program.
2. Create a socket using the appropriate transport protocol, TCP or UDP.
3. Assign an IP address and port number to the server.
4. For TCP, bind the socket, listen for connections, and accept a client connection. For UDP, bind the socket and wait for incoming datagrams.
5. Start the client program and create a corresponding socket.
6. Connect to the server for TCP or send a datagram to the server for UDP.
7. Enter a message at the client.
8. Send the message to the server.
9. The server receives the message and sends the same message back to the client.
10. The client receives and displays the echoed message.
11. Repeat the process as required and close the sockets.

PROGRAMS:

Python implementation using TCP sockets.

SERVER PROGRAM (server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)

print("TCP Echo Server is waiting for a connection...")

conn, addr = server.accept()
print("Connected to:", addr)
```

```
while True:
 data = conn.recv(1024)

if not data:
 break

message = data.decode()
 print("Received:", message)

conn.sendall(data)

conn.close()
server.close()
```

CLIENT PROGRAM (client.py)

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))

while True:
 message = input("Enter a message (exit to stop): ")

if message.lower() == "exit":
 break

client.sendall(message.encode())

response = client.recv(1024)
 print("Echo from server:", response.decode())

client.close()
```

PROCEDURE:

1. Save the server program as server.py and the client program as client.py.
2. Open two terminals in the same directory.
3. Run the server first using the command: python server.py
4. Run the client in the second terminal using the command: python client.py
5. Enter a message in the client terminal.
6. Observe that the server receives the message and echoes it back to the client.
7. Enter exit to terminate the client.

OUTPUT:

Server Terminal:

TCP Echo Server is waiting for a connection...
Connected to: ('127.0.0.1', 54321)
Received: Hello Server
Received: Computer Networks

Client Terminal:

Enter a message (exit to stop): Hello Server
Echo from server: Hello Server
Enter a message (exit to stop): Computer Networks
Echo from server: Computer Networks
Enter a message (exit to stop): exit

RESULT:

Thus, the TCP echo client-server application was implemented successfully using Python sockets. The client sent messages to the server, and the server returned the same messages, demonstrating end-to-end communication using the TCP transport protocol.